

🕒 Cập nhật tháng 8 năm 2024

[Bài đọc] UserDetailsService và PasswordEncoder

1. UserDetailsService Overview

- Là một **interface** trong Spring Security dùng để tải thông tin người dùng dựa trên tên người dùng (**username**).
- Được sử dụng trong quá trình xác thực (*authentication*) để lấy thông tin về UserDetails, bao gồm tên người dùng (**username**), mật khẩu (**password**), và vai trò/quyền (**roles**).
- Có thể được triển khai để lấy dữ liệu từ bộ nhớ (in-memory), cơ sở dữ liệu.
- Trong interface UserDetailsService cần thực thi (implement) phương thức **loadUserByUsername()** để tùy chỉnh quy trình tìm kiếm người dùng

2. Cài đặt UserDetailsService

- Cài đặt lớp User:

```
public class User {

    @Id
    @GeneratedValue(generator = "uuid2")
    @GenericGenerator(name = "uuid2", strategy = "org.hibernate.id.UUIDGenerator")
    @Column(insertable = false, updatable = false, nullable = false, columnDefinition = "CHAR(36)")
    private String id;

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private String phoneNumber;

    @Column(nullable = false)
    @JsonIgnore
    private String password;
}
```

- Tạo lớp DAO bằng Spring Data JPA bằng cách extend **JpaRepository**:

```
@Repository
public interface UserRepository extends JpaRepository<User, String> {
```

- Triển khai interface **UserDetailsService**:

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    @Transactional
    public UserDetails loadUserByUsername(String emailOrPhone) throws UsernameNotFoundException {
        User user = userRepository.findByEmailOrPhone(emailOrPhone)
            .orElseThrow(() -> new UsernameNotFoundException(String.format(ErrorMessage.User.ERR_NOT_FOUND_EMAIL_OR_PHONE, emailOrPhone)));
        return UserPrincipal.create(user);
    }

    @Override
    @Transactional
    public UserDetails loadUserById(String id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UsernameNotFoundException(String.format(ErrorMessage.User.ERR_NOT_FOUND_ID, id)));
        return UserPrincipal.create(user);
    }
}
```

3. PasswordEncoder Overview

- Là giao diện dùng để mã hóa mật khẩu và xác minh mật khẩu trong quá trình xác thực.
- Đảm bảo mật khẩu được lưu trữ an toàn dưới dạng mã hóa (ví dụ: sử dụng BCrypt).
- Các triển khai phổ biến: BCryptPasswordEncoder, Argon2PasswordEncoder, NoOpPasswordEncoder (không khuyến nghị trong production).
- Cấu hình PasswordEncoder:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

- Sử dụng passwordEncoder để mã hóa mật khẩu người dùng khi tạo:

```
@Override
public User createUser(UserCreateDTO userCreateDTO) {
    User user = new User();
    user.setPassword(passwordEncoder.encode(userCreateDTO.getPassword()));
    user.setRole(roleRepository.findByRoleName(RoleConstant.USER));
    if(userCreateDTO.getAvatarFile() != null) {
        user.setAvatar(uploadFile.uploadFile(userCreateDTO.getAvatarFile()));
    }
    return userRepository.save(user);
}
```

